GS08 · Matrículas y Notas

- **Curso GS08 · Docente: Dr. Jack Peralta**
- Arteaga Rullier, Roberto — Scrum Master + DevOps
- Díaz Cárdenas, Jorge Luis — Backend
- Inocencio Gargate, Dalcir — Frontend + Documentador
- *Presentación grupal · 21/09/2026*

El problema y el alcance del MVP

- Hoy: PHP + MySQL sin API, sin paginado, reglas dispersas en PHP, buscador LIKE que en PostgreSQL devuelve 0 filas sin avisar, y sin notas.
- Entra: autenticación, panel con KPIs, estudiantes, cursos, matrículas, usuarios, NOTAS y boleta, más contenedores, CI y Kubernetes.
- 39 criterios de aceptación con ID único (C-01…C-39): cada uno es un comando o una cifra que se puede comprobar.
- NO entra (20 puntos, por decisión): portal del estudiante, rol docente, pagos, horarios, 2FA, app móvil, nube, backups.

Arquitectura y stack

- SPA React 19 + nginx → 2 instancias del API FastAPI → PostgreSQL 16, todo en contenedores sobre una red Docker.
- FastAPI 0.141.1 · PostgreSQL 16.15 · React 19 + Vite + Tailwind · Docker 29.8.0 · Prometheus v3.13.3 + Grafana 12.4.11.
- Cinco diagramas documentados y verificados: los 5 renderizan sin error.
- Decisión central: se reimplementa el legacy, no se migra — se conservan el modelo de datos y la UX, no el código.

Modelo de datos y reglas de negocio

- 4 tablas + 1 vista traducidas de MySQL 8 a PostgreSQL 16: 16 CHECK, 6 UNIQUE, 2 FK y 7 índices.
- Las reglas RN-01…RN-16 viven en la base (última línea de defensa); la API las repite para responder 422 con mensaje.
- Semilla oficial: 12 estudiantes, 7 cursos, 24 matrículas (23 activas + 1 retirada) y 1 usuario admin.
- 54 comprobaciones del modelo y 32 de notas, con pruebas negativas que exigen que la base rechace cada regla violada: 0 fallas.

API: contrato y comportamiento

- 30 operaciones en 16 rutas, publicadas en su propio OpenAPI (versión 0.1.0): salud, autenticación, panel, estudiantes, cursos, matrículas, usuarios, notas y boleta.
- Códigos como contrato: 200/201/204 lo correcto · 401 sin sesión · 403 por rol · 404 · 409 duplicado · 422 dato inválido.
- Ningún dato del usuario puede provocar un 500: el mensaje del backend se muestra en pantalla.
- Las 30 operaciones están consumidas desde el SPA y verificadas contra la API real.

La SPA: el recorrido del usuario

- Entrar → panel con los 4 indicadores → buscar «huaman» y encontrar «Huamán» → registrar estudiante → matricular → 3 notas → ver la boleta → cursos y usuarios.
- Los errores se muestran con el mensaje del backend (409 y 422) y sin diálogos nativos del navegador.
- Menú por rol: un usuario asistente no ve ni puede llamar la pantalla de usuarios.
- Estado real: el SPA existe y consume las 30 operaciones del contrato; el recorrido se verificó de punta a punta en el navegador contra la API real.

Migración del frontend: Vue 3 → React 19

- Por qué: la rúbrica del trabajo colaborativo de Implementación de Soluciones para Plataformas Web pide React con Tailwind. Se migra el frontend para cumplirla.
- Qué cambia: solo la capa de presentación. La SPA está desacoplada del API —todo pasa por /api/v1—, así que no se toca el backend, ni la base de datos, ni la infraestructura.
- Qué se gana: de 4 vistas al consumo de las 30 operaciones del contrato, con cursos, usuarios, notas y boleta, más búsqueda, filtros, paginación y pruebas.
- Despliegue: mismo build estático servido por nginx, misma imagen y el mismo Coolify. El cambio no altera el entorno publicado: se vuelve a construir y listo.

Contenedores y CI

- 6 contenedores con un solo comando: web (nginx 8080), dos instancias del API, PostgreSQL, Prometheus y Grafana.
- Balanceo real 1 a 1 entre las dos instancias; con una apagada, 6 peticiones siguen dando 200 con reintento (failover).
- CI en cada pull request: infra + backend (ruff y pytest, cobertura mínima 60 %) + frontend + imágenes con smoke y escaneo.
- CD: publica en GHCR con SBOM y provenance; el despliegue a Kubernetes queda guardado tras DEPLOY_ENABLED=true.

Kubernetes local y observabilidad

- 11 objetos definidos y validados offline con kustomize: Namespace, ConfigMap, PVC, Job de migraciones, Deployments, Services e Ingress.
- En el cluster el esquema lo aplica Alembic con el Job de migraciones, no los scripts de inicialización.
- Observabilidad corriendo: 3 targets up y un dashboard API RED con 9 paneles con datos, provisionado desde el repositorio.
- **ESTADO REAL: el cluster NO está desplegado en esta máquina — minikube no está instalado y espera aprobación (A-02).**

Calidad: 39 criterios y lo que apareció probando

- 22 criterios verificados con corrida real · 4 incumplidos por los dos 500 · 2 en arreglo · 6 pendientes de interfaz · 5 bloqueados con motivo.
- 9 bugs documentados con pasos, esperado, obtenido y evidencia — y 4 bugs encontrados en el propio script de pruebas.
- Hallazgos que valieron la pena: un verificador que fallaba en entorno limpio, criterios con datos inexistentes y un /metrics que devolvía el SPA.
- **Regla de la casa: un bug no se cierra por decisión, se cierra con la corrida verde pegada.**

Sprint 0 → Sprint 4: entregado, bloqueado y retrospectiva

- Sprint 0 cerrado con evidencia: alcance, modelo de datos verificado, infraestructura probada, plan de pruebas y documentación completa.
- Sprint 1 en curso (28 endpoints, 22 criterios verdes) y Sprint 3 adelantado: notas y boleta funcionando.
- Bloqueado y dicho de frente: cluster de Kubernetes (A-02), repositorio público y CI, y el SPA (T2.4).
- Retrospectiva: probar temprano pagó; cada hallazgo terminó en una decisión o en una tarea con dueño.